

SQL PROJECT REAL LIFE SCENARIO

Real-world testing scenarios covering constraints, authentication reset tokens, negative tests, and test data strategies.

REGISTRATION
Constraints & Audit

FORGOT PASSWORD
Token Expiry Test

NEGATIVE SQL
Constraint Errors

DATA STRATEGY
Edge Case Matrix

1. Registration Data Validation & Constraint Testing

SCHEMA INTEGRITY

Schema enforcement requiring unique emails and non-empty passwords, followed by test record insertions.

```
CREATE TABLE users (  
  user_id INT AUTO INCREMENT PRIMARY KEY,  
  username VARCHAR(50) NOT NULL,  
  email VARCHAR(100) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  CHECK (LENGTH(TRIM(password_hash)) > 0)  
);  
  
-- Test Executions (Valid & Invalid Records)  
INSERT INTO users (username, email, password_hash) VALUES  
( 'aarav_qa', 'aarav@company.com', '$2b$12$eImiTXuWV...' ); -- Valid (Succeeds)  
  
INSERT INTO users (username, email, password_hash) VALUES  
( 'aarav_clone', 'aarav@company.com', '$2b$12$eImiTXuWV...' ); -- Duplicate Email  
  
INSERT INTO users (username, email, password_hash) VALUES  
( 'blank_pass', 'blank@company.com', ' ' ); -- Blank Password
```

Data Integrity Verification Query:

```
SELECT user_id, email, password_hash FROM users  
WHERE email IS NULL OR TRIM(password_hash) = ''  
OR email IN (SELECT email FROM users GROUP BY email HAVING COUNT(*) > 1);
```

2. E-Commerce 'Forgot Password' Active Token Filter

AUTH SIMULATION

Insert expired and valid reset tokens, then query to isolate active requests.

```
INSERT INTO password_resets (user_id, reset_token, expires_at) VALUES  
(101, 'tok_expired_999', NOW() - INTERVAL '1 hour'), -- Expired Token  
(102, 'tok_valid_123', NOW() + INTERVAL '15 minute'); -- Active Token  
  
SELECT reset_id, user_id, reset_token, expires_at  
FROM password_resets  
WHERE expires_at > CURRENT_TIMESTAMP;
```

Active Reset Tokens Output:

reset_id	user_id	reset_token	expires_at
2	102	tok_valid_123	2026-09-23 00:41:31

3. Negative SQL Testing on product_orders Table

NEGATIVE TESTING

Attempt inserting an order referencing a non-existent product and a negative quantity.

```
INSERT INTO product_orders (product_id, quantity) VALUES (99999, -5);
```

CHECK CONSTRAINT ERROR (QUANTITY <= 0)

ERROR 3819 (HY000): Check constraint 'chk_positive_qty' is violated.

FOREIGN KEY CONSTRAINT ERROR (INVALID PRODUCT_ID)

ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails.

4. Safe Test Data Cleanup Script

MAINTENANCE SCRIPT

Transactional cleanup removing test accounts (@test.com) while keeping real accounts intact.

```
START TRANSACTION;

-- Delete child records in dependent tables first
DELETE FROM orders
WHERE user_id IN (SELECT user_id FROM users WHERE email LIKE '%@test.com');

-- Delete test users
DELETE FROM users WHERE email LIKE '%@test.com';

COMMIT;
```

5. Test Data Strategy for User Order History

EDGE CASE DESIGN

Generation strategy covering edge cases for user order history features.

```
-- Insert 5 Users (Including 1 User with 0 Orders)
INSERT INTO users (user_id, username, email) VALUES
(1, 'usr_multi', 'u1@test.com'), (2, 'usr_two', 'u2@test.com'),
(3, 'usr_recent', 'u3@test.com'), (4, 'usr_old', 'u4@test.com'),
(5, 'usr_zero', 'u5@test.com'); -- Edge Case: No Orders
```

Target User	Data Profile	Quality Assurance Goal / Edge Case
User 5	0 Orders	Verifies UI empty state rendering ("No orders found")
User 1	3 Orders	Tests list sorting (ORDER BY order_date DESC) & pagination
User 3 & 4	Recent & Old Orders	Tests date range filters (e.g., current year vs past years)